

Corrigé - Examen Ingénierie Logicielle (HAI721) - Session 1 (2022-2023)

A. Framework - Architectures extensibles (6 points)

Question 1 : Architecture logicielle

Pattern Composite.

```

+-----+
|          ComposantOrdi (abstract)          |
+-----+
|          #TVA : double                      |
|          #getTVA() : double                 |
|          #prixHT() : double (abstract)      |
|          +prixTTC() : double                |
+-----+
|          / \                               |
+-----+ +-----+
|          ComposantSimple (abstract)         | |          Montage
+-----+ +-----+
|          / \                               | | -composants : List<ComposantOrdi>
|          |                               | | +add(ComposantOrdi c)
+-----+ +-----+
|          RAM                               | |          #prixHT() : double
+-----+ +-----+
|          -prixBase : double                 |
|          #prixHT() : double                 |
+-----+

```

Question 2 : Fonction d'ordre supérieur et liaison dynamique

`prixTTC()` est une fonction d'ordre supérieur car elle délègue à `prixHT()` (abstraite).

La liaison dynamique : quand `prixTTC()` appelle `prixHT()`, c'est la méthode de la classe réelle de l'objet (type dynamique) qui est exécutée.

Question 3 : Définition de la classe Montage

```

public class Montage extends ComposantOrdi {
    private List<ComposantOrdi> composants = new ArrayList<>();

    public void add(ComposantOrdi c) {
        composants.add(c);
    }

    @Override
    protected double prixHT() {
        double total = 0;
        for (ComposantOrdi c : composants) {
            total += c.prixHT();
        }
        return total;
    }
}

```

Question 4 : Points d'extensions et inversions de contrôle

Points d'extensions : Méthodes abstraites `prixHT()`, création de sous-classes.

Inversion de contrôle : Framework contrôle le flux (`prixTTC()` appelle `prixHT()` polymorphiquement).

Injection de dépendances : Via `add(ComposantOrdi c)` dans `Montage`.

Question 5 : Fluent interface

```

public Montage add(ComposantOrdi c) {
    composants.add(c);
    return this;
}

```

B. Amélioration de l'architecture (1) - Cohérence des montages (3 points)

Question 1 : Pattern Fabrique Abstraite

```

public interface FabriqueMontage {
    Montage createMontage(char config);
}

public class FabriqueMontagePredefini1 implements FabriqueMontage {
    @Override
    public Montage createMontage(char config) {
        Montage m = new MontagePredefini1();
        if (config == 'A') {
            m.add(new RAM("RAM"));
        }
    }
}

```

```

        return m;
    }
}

```

C. Extensibilité et Typage Statique (6 points)

Question 1 : Choix de signature pour `equiv()`

a) Oui, les deux compilent séparément :

- `boolean equiv(Montage c, String critere)` : surcharge
- `boolean equiv(ComposantOrdi c, String critere)` : redéfinition

b) Choix : `boolean equiv(ComposantOrdi c, String critere)`

Respecte polymorphisme, résolution dynamique, évite casts.

Question 2 : Analyse des appels

```

RAM m1 = new RAM();
Montage m2 = new Montage();
ComposantOrdi m3 = m1;
ComposantOrdi m4 = m2;
m2.equiv(m4, "x");           // 5 → Montage.equiv
m3.equiv(m4, "x");           // 6 → ComposantOrdi.equiv
m4.equiv(m4, "x");           // 7 → Montage.equiv
m4.equiv((Montage)m4, "x"); // 8 → Montage.equiv
m4.equiv(m3, "x");           // 9 → Montage.equiv

```

Explication :

- **5** : Receveur statique/dynamique `Montage` → `Montage.equiv`
- **6** : Receveur statique `ComposantOrdi`, dynamique `RAM` → `ComposantOrdi.equiv` (pas de redéfinition dans `RAM`)
- **7** : Receveur dynamique `Montage` → liaison dynamique → `Montage.equiv`
- **8** : Cast argument, surcharge non applicable → redéfinition via dynamique → `Montage.equiv`
- **9** : Receveur dynamique `Montage` → `Montage.equiv`

Concepts : Typage statique détermine surcharge (choix méthode par types arguments à compilation). Liaison dynamique résout redéfinition (override) à l'exécution via type dynamique receveur. Polymorphisme d'inclusion permet variable type base référence sous-classe.

Question 3 : Commentaire sur `ComposantOrdi m4 = m2;`

Polymorphisme d'inclusion. Principe Ouvert/Fermé : code client via interface abstraite, extensible sans modification.

Question 4 : Solution sans `instanceof` (Double Dispatch)

Points clés du code :

1. Ajouter méthode `equivTo` dans `ComposantOrdi` :

```
public boolean equiv(ComposantOrdi c, String crit) {  
    return c.equivTo(this, crit); // Double dispatch  
}  
  
protected abstract boolean equivTo(ComposantOrdi other, String crit);
```

2. Sous-classes redéfinissent `equivTo` pour cas spécifiques (ex: `RAM.equivTo` compare avec autre `RAM`).

Principe : Inversion : l'argument "visite" le receveur via `equivTo`, évite tests explicites dans `Montage.equiv`.

D. Amélioration de l'architecture (2) (5 points)

Question 1 : Comparaison State vs Decorator

State : États mutables, moins adapté variations additives.

Decorator : Comportements dynamiques en couches, combinaisons multiples.

Choix : Decorator (variations cumulatives, extensibilité).

Question 2 : Éléments clés du code

Classes clés :

```
public abstract class DecoratorPrix extends ComposantOrdi {  
    protected ComposantOrdi wrapped;  
  
    public DecoratorPrix(ComposantOrdi w) {  
        wrapped = w;  
    }  
  
    @Override  
    protected double prixHT() {  
        return wrapped.prixHT() + calculVariation();  
    }  
  
    protected abstract double calculVariation();  
}  
  
public class VariationTransport extends DecoratorPrix {  
    private double facteurTransport;  
  
    public VariationTransport(ComposantOrdi w, double facteur) {  
        super(w);  
        this.facteurTransport = facteur;  
    }  
}
```

```

    @Override
    protected double calculVariation() {
        return wrapped.getPoids() * facteurTransport;
    }
}

public class VariationMatiere extends DecoratorPrix {
    private double facteurMatiere;

    public VariationMatiere(ComposantOrdi w, double facteur) {
        super(w);
        this.facteurMatiere = facteur;
    }

    @Override
    protected double calculVariation() {
        return wrapped.prixHT() * facteurMatiere;
    }
}

```

Envois de messages clés :

- Création : `new VariationTransport(new VariationMatiere(new RAM(), 0.1), 0.5)`
- Calcul : `ramVariee.prixTTC()` → appelle `prixHT()` décorateur externe → somme variations + `wrapped.prixHT()` (récursif jusqu'à feuille)

Méthode(s) `prixHT()` :

```

// Dans DecoratorPrix
@Override
protected double prixHT() {
    return wrapped.prixHT() + calculVariation();
}

```